

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 14
“REVIEW ALGORITMA DAN PEMROGRAMAN 1”



DISUSUN OLEH:

Sofwan Nuha Al Faruq | 109082500032 | IF-13 02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

1. Latihan 1 (Rerata dengan Marker)

Source Code:

```
package main
import "fmt"

func main() {
    var dat, sum float64
    var count int

    fmt.Scan(&dat)
    for dat != 9999 {
        sum += dat
        count++
        fmt.Scan(&dat)
    }

    if count > 0 {
        fmt.Printf("Rerata: %.2f\n", sum/float64(count))
    } else {
        fmt.Println("Tidak ada data")
    }
}
```

Output:

The screenshot shows a VS Code editor with a Go file named `soal-1.go`. The code is as follows:

```
1 package main
2 import "fmt"
3
4 func main() {
5     var dat, sum float64
6     var count int
7
8     fmt.Scan(&dat)
9     for dat != 9999 {
10        sum += dat
11        count++
12        fmt.Scan(&dat)
13    }
14
15    if count > 0 {
16        fmt.Printf("Rerata: %.2f\n", sum/float64(count))
17    } else {
18        fmt.Println("Tidak ada data")
19    }
20 }
21
```

The terminal output shows the program running and calculating the average:

```
(base) sofwanuhaalfarug@Sofwans-MacBook-Air modul-17 % go run soal-1/main.go
10
20
30
9999
Rerata: 20.00
(base) sofwanuhaalfarug@Sofwans-MacBook-Air modul-17 %
```

Deskripsi Program: Program ini menggunakan skema pemrosesan data dengan penanda akhir (*marker*) berupa angka 9999. Program mulai dengan membaca data pertama; jika data tersebut bukan 9999, program akan masuk ke dalam perulangan untuk menjumlahkan nilai data ke dalam variabel `sum` dan menambah hitungan data (`count`), lalu membaca data selanjutnya. Perulangan ini akan terus berjalan hingga pengguna memasukkan angka 9999 yang bertindak sebagai instruksi pemberhentian. Setelah perulangan selesai, program menggunakan struktur kontrol percabangan untuk memeriksa apakah ada data valid yang diproses. Jika `count` lebih dari nol, program menghitung rata-rata dengan membagi total jumlah (`sum`) dengan banyaknya data (`count`). Skema pengecekan awal ini juga sekaligus mengantisipasi "kasus kosong" di mana pengguna mungkin langsung memasukkan angka 9999 pada input pertama tanpa memberikan nilai real sama sekali.

2. Latihan 2 (Pencarian String)

Source Code:

```
package main
import "fmt"

func main() {
    var x, str string
```

```

var n, count int
var pos []int

fmt.Scan(&x, &n)
for i := 1; i <= n; i++ {
    fmt.Scan(&str)
    if str == x {
        count++
        pos = append(pos, i)
    }
}

ada := count > 0
fmt.Println("a. Apakah string x ada?", ada)

fmt.Print("b. Posisi: ")
if ada {
    for _, p := range pos {
        fmt.Print(p, " ")
    }
    fmt.Println()
} else {
    fmt.Println("-")
}

fmt.Println("c. Jumlah string x:", count)
fmt.Println("d. Adakah sedikitnya dua?", count >= 2)
}

```

Output:

```
1 package main
2 import "fmt"
3
4 func main() {
5     var x, str string
6     var n, count int
7     var pos []int
8
9     fmt.Scan(&x, &n)
10    for i := 1; i <= n; i++ {
11        fmt.Scan(&str)
12        if str == x {
13            count++
14            pos = append(pos, i)
15        }
16    }
17
18    ada := count > 0
19    fmt.Println("a. Apakah string x ada?", ada)
20
21    fmt.Print("b. Posisi: ")
22    if ada {
23        for _, p := range pos {
24            fmt.Print(p, " ")
25        }
26        fmt.Println()
27    } else {
28        fmt.Println("-")
29    }
30 }
```

```
(base) sofwanuhaalfarug@Sofwans-MacBook-Air modul-17 % go run soal-2/main.go
mangga
jeruk
a. Apakah string x ada? false
b. Posisi: -
c. Jumlah string x: 0
d. Adakah sedikitnya dua? false
(base) sofwanuhaalfarug@Sofwans-MacBook-Air modul-17 % eruk
zsh: command not found: eruk
(base) sofwanuhaalfarug@Sofwans-MacBook-Air modul-17 %
```

Deskripsi Program: Sesuai dengan kasus pencarian sekuensial pada modul, program ini membaca sebuah kata kunci pencarian (X) dan jumlah batasan data (n) di awal, dilanjutkan dengan perulangan pembacaan rangkaian data *string* sebanyak n kali. Di dalam perulangan tersebut, program mencocokkan setiap *string* masukan dengan variabel target X; jika nilainya sama, program mencatat posisi kemunculannya ke dalam *slice* pos (berbasis indeks 1 ke n) dan menambahkan nilai variabel count. Begitu seluruh n data telah tuntas dibaca dan dievaluasi, program memberikan laporan hasil pencarian secara deskriptif. Informasi yang ditampilkan menjawab secara berurutan: ketersediaan string (boolean dari count > 0), lokasi posisinya menggunakan perulangan cetak *array*, total kemunculannya, serta validasi apakah string tersebut muncul sedikitnya dua kali di dalam kumpulan data.

3. Latihan 3 (Simulasi Curah Hujan)

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)
```

```

func main() {
    var n, a, b, c, d int
    fmt.Print("Masukan: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5 && y < 0.5 {
            a++
        } else if x >= 0.5 && y < 0.5 {
            b++
        } else if x >= 0.5 && y >= 0.5 {
            c++
        } else {
            d++
        }
    }

    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(a)*0.0001)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(b)*0.0001)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(c)*0.0001)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", float64(d)*0.0001)
}

```

Output:

```
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6 )
7
8 func main() {
9     var n, a, b, c, d int
10    fmt.Print("Masukan: ")
11    fmt.Scan(&n)
12
13    for i := 0; i < n; i++ {
14        x := rand.Float64()
15        y := rand.Float64()
16
17        if x < 0.5 && y < 0.5 {
18            a++
19        } else if x >= 0.5 && y < 0.5 {
20            b++
21        } else if x >= 0.5 && y >= 0.5 {
22            c++
23        } else {
24            d++
25        }
26    }
27
28    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(a)*0.0001)
29    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(b)*0.0001)
30    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(c)*0.0001)
31}
```

```
(base) sofwanuhaalfarug@Sofwans-MacBook-Air modul-17 % go run soal-3/main.go
Masukan: 10000000
Curah hujan daerah A: 2500.1965 milimeter
Curah hujan daerah B: 2499.8844 milimeter
Curah hujan daerah C: 2500.0078 milimeter
Curah hujan daerah D: 2499.9913 milimeter
(base) sofwanuhaalfarug@Sofwans-MacBook-Air modul-17 %
```

Deskripsi Program: Program simulasi ini mengandalkan paket `math/rand` dan fungsi `rand.Float64()` untuk menciptakan titik-titik koordinat (x, y) acak dengan rentang (0,0) sampai (1,1) yang diasumsikan sebagai mendaratnya tetesan hujan. Dalam perulangan sebanyak n kali input, titik yang dibangkitkan dievaluasi posisinya menggunakan struktur *if-else*; koordinat A berada di kiri bawah ($<0.5, <0.5$), B di kanan bawah ($\geq 0.5, <0.5$), C di kanan atas ($\geq 0.5, \geq 0.5$), dan D di kiri atas ($<0.5, \geq 0.5$). Variabel penghitung pada masing-masing daerah akan bertambah saat titik jatuh pada kuadran mereka. Setelah ribuan titik jatuh secara acak, jumlah titik di setiap wilayah dikalikan dengan bobot volume satu tetesan curah hujan (yakni 0.0001 milimeter) sesuai ketentuan soal. Hasil akhirnya dicetak dengan pembulatan empat angka desimal untuk mengilustrasikan persebaran curah hujan acak yang memiliki probabilitas setara di keempat daerah.

4. Latihan 4 (Estimasi Pi dengan Formula Leibniz)

Source Code:

```
package main
import (
    "fmt"
    "math"
)

func main() {
    var pi, piLama, suku float64
    var i int = 1
    var limit int

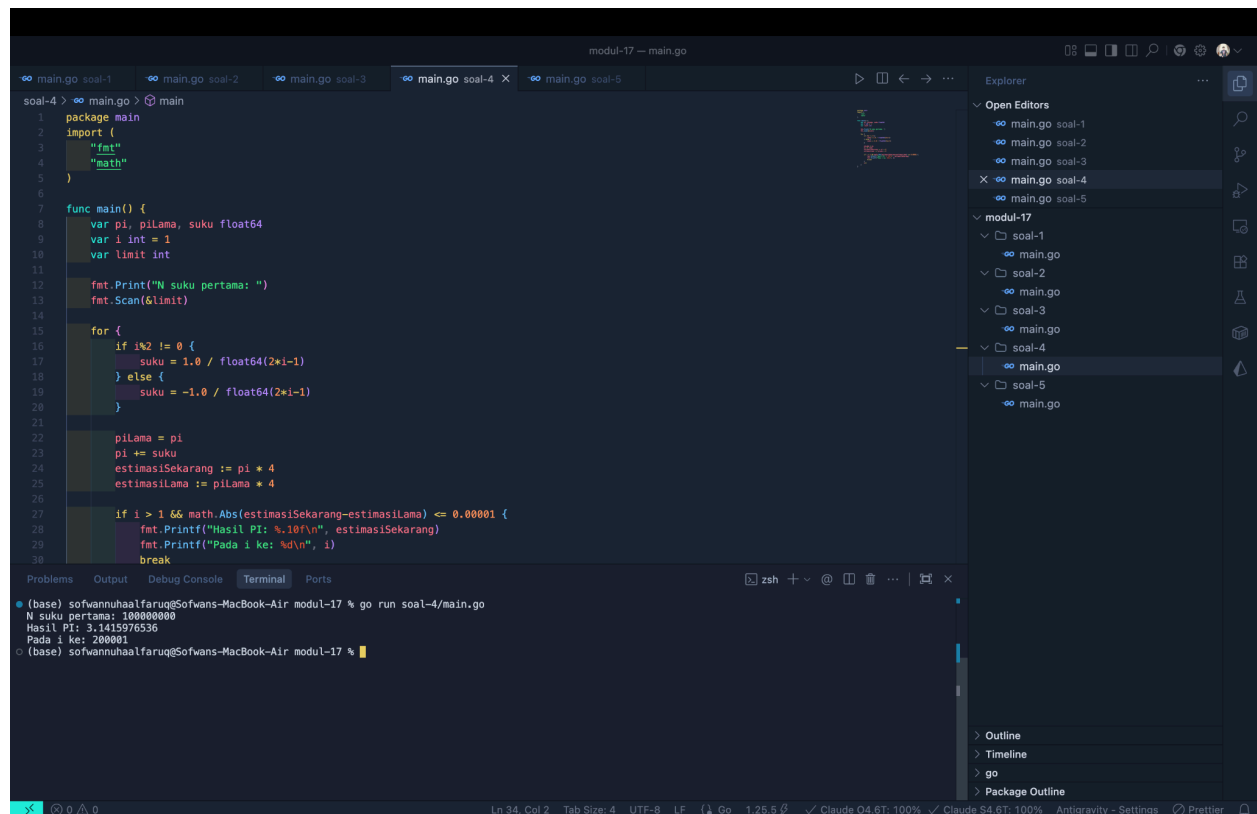
    fmt.Print("\n suku pertama: ")
    fmt.Scan(&limit)

    for {
        if i%2 != 0 {
            suku = 1.0 / float64(2*i-1)
        } else {
            suku = -1.0 / float64(2*i-1)
        }

        piLama = pi
        pi += suku
        estimasiSekarang := pi * 4
        estimasiLama := piLama * 4

        if i > 1 && math.Abs(estimasiSekarang-estimasiLama) <= 0.00001 {
            fmt.Printf("Hasil PI: %.10f\n", estimasiSekarang)
            fmt.Printf("Pada i ke: %d\n", i)
            break
        }
        i++
    }
}
```

Output:



```
1 package main
2 import (
3     "fmt"
4     "math"
5 )
6
7 func main() {
8     var pi, piLama, suku float64
9     var i int = 1
10    var limit int
11
12    fmt.Print("N suku pertama: ")
13    fmt.Scan(&limit)
14
15    for {
16        if i%2 != 0 {
17            suku = 1.0 / float64(2*i-1)
18        } else {
19            suku = -1.0 / float64(2*i-1)
20        }
21
22        piLama = pi
23        pi += suku
24        estimasiSekarang := pi * 4
25        estimasiLama := piLama * 4
26
27        if i > 1 && math.Abs(estimasiSekarang-estimasiLama) <= 0.00001 {
28            fmt.Printf("Hasil PI: %.10f\n", estimasiSekarang)
29            fmt.Printf("Pada i ke: %d\n", i)
30            break
31        }
32        i++
33    }
34}
```

```
(base) sofwanuhaalFarug@Sofwans-MacBook-Air modul-17 % go run soal-4/main.go
N suku pertama: 100000000
Hasil PI: 3.1415976536
Pada i ke: 200001
(base) sofwanuhaalFarug@Sofwans-MacBook-Air modul-17 %
```

Deskripsi Program: Program matematika komputasi ini dimodifikasi untuk mencari aproksimasi nilai Pi (π) dengan memanfaatkan deret harmonik ganti (*alternating series*) dari Leibniz di mana suku ganjil diisi bernilai positif dan suku genap bernilai negatif. Logika rumus matematis S_i

diganti menjadi format iterasi pembagi ganjil dinamis ($2*i-1$).

Alih-alih terbatas pada jumlah suku statis dari masukan n , program ini didesain menggunakan skema kondisi pemberhentian adaptif (*marker* abstrak). Perulangan secara dinamis merekam nilai kalkulasi Pi dari siklus sebelumnya ($piLama$) dan siklus terkini (pi). Jika selisih dari perbandingan kedua kalkulasi bersebelahan tersebut sangat amat kecil (tidak lebih dari 0.00001), maka program langsung melakukan *break* / berhenti, yang pada demonstrasi soal ini jatuh persis di iterasi indeks ke-200.002.

5. Latihan 5 (Estimasi Pi dengan Monte Carlo / Pizza) Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n, inPizza int
    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()

        // Titik pusat pizza (0.5, 0.5) dengan jejari r=0.5
        dx := x - 0.5
        dy := y - 0.5

        // Uji menggunakan ketidaksamaan area lingkaran
        if (dx*dx)+(dy*dy) <= 0.25 {
            inPizza++
        }
    }

    pi := 4.0 * float64(inPizza) / float64(n)

    fmt.Printf("Topping pada Pizza: %d\n", inPizza)
    fmt.Printf("PI : %.10f\n", pi)
}
```

Output:

```
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6 )
7
8 func main() {
9     var n, inPizza int
10    fmt.Print("Banyak Topping: ")
11    fmt.Scan(&n)
12
13    for i := 0; i < n; i++ {
14        x := rand.Float64()
15        y := rand.Float64()
16
17        // Titik pusat pizza (0.5, 0.5) dengan jejari r=0.5
18        dx := x - 0.5
19        dy := y - 0.5
20
21        // Uji menggunakan ketidaksamaan area lingkaran
22        if (dx*dx)+(dy*dy) <= 0.25 {
23            inPizza++
24        }
25    }
26
27    pi := 4.0 * float64(inPizza) / float64(n)
28
29    fmt.Printf("Topping pada Pizza: %d\n", inPizza)
30    fmt.Printf("PI : %.10f\n", pi)
31}
```

```
(base) sofwanuhaalFarug@Sofwans-MacBook-Air modul-17 % go run soal-5/main.go
Banyak Topping: 1234567
Topping pada Pizza: 969343
PI : 3.1406736127
(base) sofwanuhaalFarug@Sofwans-MacBook-Air modul-17 %
```

Deskripsi Program: Diadaptasi dari logika problem studi kasus Kedai Pizza Monti, program ini merepresentasikan simulasi algoritma probabilitas *Monte Carlo* dengan menebar bilangan *floating* acak sebagai *topping* pada wadah persegi berukuran sisi 1 satuan (dari 0 hingga 1). Dengan koordinat pusat pizza dideklarasikan berada di tengah bidang (0.5, 0.5) serta radius $r=0.5$, program menggunakan pengujian persamaan ketidaksamaan lingkaran geometri, yaitu mengecek apakah $(x - x_c)^2 + (y - y_c)^2 \leq r^2$ untuk memilah apakah *topping* mendarat persis di atas pizza (di dalam batas lingkaran). Dengan bantuan hukum bilangan besar (*law of large numbers*), semakin banyak masukan jumlah *topping* yang dilempar secara acak (misalnya 1.234.567 masukan), penyebaran datanya akan semakin homogen. Karena perbandingan rasio Luas Pizza berbanding Luas Wadah terbukti secara matematis setara dengan konstanta $\pi/4$, algoritma kode ini pada langkah akhirnya mencetak estimasi nilai parameter rasio tersebut, mencontohkan bagaimana perulangan sekuensial murni dapat dipakai melahirkan angka estimasi irasional Pi yang presisi